EXPERIMENT NO: 1

TITLE: Develop a program to find maximum and minimum elements present in a list.

AIM:

To implement a program in Python that:

- finds the maximum element in a list.
- finds the minimum element in a list.

SOFTWARE REQUIRED: Python (IDLE and IDE)

THEORY:

A list is a data structure that holds an ordered collection of items, it allows for dynamic sizing and supports operations like insertion, deletion, and retrival of elements.

1) Finding a maximum Element:

a. Iterate through the list and compare each element with the current maximum value.

b. update the maximum value when a larger element is found.

2) Finding minimum Element:

a. Iterate through list and compare each element with the current minimum value.

b. update the minimum value when a smaller element is found.

3) Time complexity:

a. Best case = O(n) → iterate through all elements once

b. worst case: O(n) → same as the best case. since all elements are scanned

ALGORITHM:

1) Start
2) Create a list with integer element
3) Initialize two variables:
   a. max_element → set to the first element of the list
   b. min_element → set to the first element of the list
4) Traverse the list using a loop:
   a. If the current element is greater than max_element:
      i. update max_element
   b. If the current element is smaller than min_element:
      i. update min_element
5) Display the maximum and minimum elements
6. Stop.

Program:

```
l = []
n = int(input("Enter total number of elements: "))
for i in range(n):
    l.append(int(input(f"Enter number {i+1}: ")))

max = l[0]
min = l[0]
for i in range(n):
    if l[i] > max:
        max = l[i]
    if l[i] < min:
        min = l[i]
print(f"max: {max}, min: {min}")
```

Output:

Enter total number of elements: 5
Enter number 1: 66
Enter number 2: 81
Enter number 3: 10
Enter number 4: 9
Enter number 5: 20
max: 81 , min = 9

CONCLUSION:

The experiment successfully demonstrates:

- Finding the maximum and minimum elements in a list using python
- Iterating through the list efficiently with O(n) time complexity

EXPERIMENT NO: 2

TITLE: Develop a Program to find second largest and second smallest element present in the list.

AIM:

To implement a program in Python that:

- finds the second largest element in a list
- finds the second smallest element in a list

SOFTWARE REQUIREMENT: Python IDE

THEORY:

- A list is a data structure that holds an ordered collection of items. It allows for dynamic sizing and supports operations like insertion, deletion, and retrieval of elements.

1) Finding the seand largest element:
   a. Traverse the list and find the largest element.
   b. During the same traversal, keep track of the secons largest element.
2) Finding the second smallest element:
   a Traversal the list and find the smallest element.
   b. During the same traversal, keep track of the seand smallest element.
3) Time complexity:
   a Best case: O(n) → Iterate through all elements once
   b worst case: O(n) → same as Best case

ALGORITHM:

1) Start
2) Create a list with integer elements.
3) Initialize variables:
   - Largest → set of the first elements.
   - second largest → set to None.
   - smallest → set to the first element
   - second → smallest → set to None.
4) Traverse the list
- find the max or largest element and store max in second min
- find the min or smallest element and store min in second max
- use the looping condition for numbers in range.
- if the list element is greater than second max and not equal to max then second max assign.
- elif the list element is less than second min and not equal to min then second min assign.
5) Print the second max and second min
6) Stop

Program:

```
l=[]
n=int(input("Enter total number of elements:"))
for i in range(n):
    l.append(int(input(f"Enter number {i+1}:")))

max=l[0]
min=l[0]
```

```
for i in range (n):
    if l[i] > max:
        max = l[i]
    if l[i] < min:
        min = l[i]

smax = min
smin = max
for i in range (n):
    if l[i] > smax and l[i] != max:
        smax = l[i]
    if l[i] < smin and l[i] != min:
        smin = l[i]
print (f" second max = {smax} & second min = {smin}")
```

Output:

```
Enter total number of elements = 5
Enter number 1 : 11
Enter number 2 : 22
Enter number 3 : 9
Enter number 4 : 4
Enter number 5 : 41
Second max = 22 & second min = 9
```

CONCLUSION:

The experiment successfully demonstrates:

- Finding the second largest and second smallest elements in a list.
- Iterating through the list efficiently with O(n) time complexity.

EXPERIMENT NO: 3

TITLE: Insertion and Deletion operations on Singly Linked List

AIM:

To implement a program in Python that:

- Performs insertion operation (at beginning, end, and specific position) on a singly linked list
- Performs deletion operation (at beginning, end, and specific position) on a singly linked list.

SOFTWARE REQUIRED: Python IDE

THEORY:

- A singly linked list (SLL) is a data structure consisting of nodes. Each node contains:
  - Data → Holds the actual value
  - Next pointer → Points to the next node in the list.

1 Insertion operations:

- At the Beginning:
  - Create a new node
  - Point the new node's next to the current head.
  - Update the head to the new node.
- At the End:
  - Traverse the list until the last node
  - Set the last node's next to the new node.
- At a specific position:
  - Traverse to the desired position.
  - Insert the new node by adjusting the next pointers.

2) Deletion operations:

- From the Beginning:
  - Change the head to the next node
  - Delete the current head node.
- From the End
  - Traverse the list until the second last node
  - set its next to None
  - Delete the last node
- From a specific position
  - Traverse to the desired position.
  - Adjust the next pointers to remove the target node

3) Time complexity:

- Insertion at Beginning : O(1)
- Insertion at end : O(n)
- Insertion at specific position : O(n)
- deletion at beginning : O(1)
- deletion at end : O(n)
- deletion at specific position : O(n)

ALGORITHM:

Insertion in singly linked list:

1) start
2) Create a Node:
   a. create a new node with the given data.
3) Insertion at Beginning:
   a. set the next pointer of the new node to the current head.

b. update the head to the new node

4) Insertion at end
   a. Traverse the list till the last node
   b. set the next pointer of the last node to the new node.

5) Insertion at specific position
   a. Traverse to the desired position
   b. Adjust the next pointers to insert the new node

6) Display the linked list

7) Stop

Deletion in singly linked list:

1) Start

2) Check if the list is empty:
   a. if head == None, print "list is empty" and stop.

3) Deletion from Beginning:
   a. set the head to the next node.

4) Deletion from End:
   a. Traverse the list until the second last node.
   b. Set its next to None.

5) Deletion from specific position.
   a. Traverse to the previous node of the target position
   b. Adjust the next pointers to bypass the target node.

6) Display the linked list

7) Stop

PROGRAM:

```
Class Node:
    def __init__(self, data, next = None):
        self.data = data
        self.next = next
Class LL:
    def __init__(self):
        self.head = None
    def insert_at_beginning(self, node):
        node.next = self.head
        self.head = none
    def insert_at_last(self, node):
        temp = self.head
        if temp:
            while temp.next != None:
                temp = temp.next
            temp.next = None
        else:
            self.insert_at_beginning(node)
    def display(self):
        if self.head == None:
            print("Linked list is empty")
        else:
            temp = self.head
            while temp != None:
                print(temp.data)
                temp = temp.next
    def insert_after(self, num, node):
```

```
        p = self.head
        if not p:
            self.insert_at_beginning(node)
        elif:
            while p != None and p.data != num:
                p = p.next
            if p:
                node.next = p.next
                p.next = node
            elif:
                print("No such data found")
    def del_at_beginning(self):
        t = self.head
        if t:
            self.head = self.head.next
            del t
    def del_at_last(self):
        p = None
        t = self.head
        if t:
            while t.next != None:
                p = t
                t = t.next
            p.next = None
            del t
            print("Successfully deleted!")
    def del_num(self, num):
        p = None
        t = self.head
```

```
        if t.data==num:
            self.head = t.next
            del t
        elif t:
            while t!= None and t.data != num:
                p=t
                t = t.next
            if t == None:
                print ("data not found")
            else:
                p.next = t.next
                del t
                Print ("successfully deleted")
ll=LL()
while 1:
    Print ("Press 1 to insert at beginning")
    Print ("press 2 to insert at last")
    Print ("Press 3 to insert after specific data")
    Print ("press 4 to delete at beginning")
    Print ("Press 5 to delete at last")
    Print ("press 6 to delete particular data")
    Print ("Press 7 to display")
    print ("Press 8 to exit")

    choice = int (input (" "))
    if choice > 0 and Choice < 4:
        data = int (" input (" Enter the number"))
        Node = Node (data)
        if choice == 1:
```

```
            ll.insert_at_beginning(node)
        elif choice==2:
            ll.insert_at_last(node)
        else:
            num=int(input("Enter the number after which you have to insert"))
            ll.insert_after(num,node)
    elif choice>3 and choice<7:
        if choice==4:
            ll.del_at_beginning()
        elif choice==5:
            ll.del_at_last()
        else:
            num=int(input("Enter the no which you want to delete"))
            ll.del_num(num)
    elif choice==7:
        ll.display()
    elif choice==8:
        print("Exiting the program")
        break
    else:
        print("Invalid choice, please try again")
```

Output:

press 1 to insert at beginning
press 2 to insert at last
press 3 to insert after specific data
press 4 to delete at beginning
press 5 to delete at last
press 6 to delete particular data

EXPERIMENT NO: 4

TITLE: Implementation of stack and its operations (Push, pop, Peek)

AIM: To implement a program in Python that:

- Performs push operation → Adds an elements to the top of the stack.
- Performs Pop operation → Remove the topmost element from the stack.
- Performs Peek operation → Displays the topmost element of the stack.

SOFTWARE REQUIRED → Python IDE

THEORY:

- A Stack is a linear data structure that stores elements in a LIFO manner.
- LIFO Principle:
  - The last element inserted is the first one to be removed.
  - Examples: A stack of Plates - the plate added last it removed first.

1) Stack operations:
   - Push: Adds an element to the top of the stack
   - POP: Removes the topmost element.
   - Peek: Displays the topmost element without removing it.
2) Time complexity:
   - Push: O(1) → Inserting at the top takes constant time.
   - POP: O(1) → Removing the topmost element takes constant time
   - Peek: O(1) → Accessing the top element takes constant time
3) Applications of Stack:
   - Undo operations in text editors.
   - Function call management in recursion
   - Backtracking in algorithms like DFS.

ALGORITHM:

Push operations

1) Start
2) Check if the stack is full:
   a. If yes, print "stack overflow" and stop.
   b. Else, proceed to the next step.
3) Insert the element at the top of the stack.
4) Increase the top index by 1.
5) Stop.

Pop operations

1) start
2) Check if the stack is empty:
   a. If yes, print "stack underflow" and stop.
   b. Else, proceed to the next step.
3) Remove the topmost element
4) Decrease the top index by 1.
5) Display the popped element
6) stop

Peek operation:

1) Start
2) Check if the stack is empty:
   a. If yes, print "stack is empty" and stop.
   b. Else, proceed to the next step.
3) Display the topmost element
4) Stop

PROGRAM:

```
Class Stack:
    def __init__(self):
        self.stack = []
    def push(self, item):
        self.stack.append(item)
        print(f"pushed {item} onto the stack.")
    def pop(self):
        if self.is_empty():
            print("stack is empty. cannot pop.")
            return None
        item = self.stack.pop()
        print(f"Popped {item} from the stack.")
        return item
    def peek(self):
        if self.is_empty():
            print("stack is empty. Nothing to peek")
            return None
        print(f"Top of the stack is {self.stack[-1]}")
        return self.stack[-1]
    def is_empty(self):
        return len(self.stack)==0
    def display(self):
        print("current stack:", self.stack)

S = Stack()
S.push(10)
S.push(20)
S.push(30)
```

```
S.display()
S.peek()
S.pop()
S.display()
```

Output:

```
Pushed 10 onto the stack.
Pushed 20 onto the stack
Pushed 30 onto the stack
Current stack: [10, 20, 30]
Top of the stack is 30
Popped 30 from the stack.
Current stack : [10, 20]
```

CONCLUSION:

The experiment successfully demonstrates:

- Push operation → Adding an element to the stack
- Pop operation → Removing the topmost element.
- Peek operation → Displaying the topmost element without removing it.
- Understanding the time complexity of stack operations O(1) for Push, Pop, and Peek.

EXPERIMENT NO: 5

TITLE: Implement Infix to Postfix conversion using stack.

AIM:

To implement a Program in Python that:

- Convert the infix expression into a Postfix expression using the stack data structure.

SOFTWARE REQUIRED: Python (IDLE or any IDE)

THEORY:

Expression Notations:

- Infix Notations: The operator is placed between the operands.
  - ex: A+B
- Postfix Notations: The operator is placed after the operands.
  - ex: AB+

Why use Postfix Notation?

- Postfix expression do not require parentheses or precedence rules.
- They are easier to evaluate using stacks.

Stack usage in conversion:

1) operands:
   - Directly added to the postfix expression.
2) operators:
   - Pushed onto the stack according to precedence and associativity.
3) Parenthesis:
   - Left Parenthesis (→ Pushed onto the stack.
   - Right Parenthesis) → Pop from the stack until a left parenthesis is found.

Operator Precedence:

- ^ → highest precedence
- * and / → highest precedence
- + and - → lowest precedence

ALGORITHM:

1) Start
2) Input and the infix expression.
3) Initialize an empty stack and an empty postfix expression.
4) Traverse the infix expression:
   - If the character is an operand (A-Z or 0-9):
     - Append it to the Postfix expression
   - If the character is a left parenthesis (:
     - Push it onto the stack.
   - If the character is a right parenthesis):
     - Pop from the stack and append to the postfix expression until a left parenthesis is encountered.
     - Remove the left parenthesis from the stack.
   - If the character in an operator.
     - While the stack is not empty and the precedence of the operator on top of the stack is greater than or equal to the current operator.
       - Pop from the stack and append to the postfix expression
     - Push the current operator onto the stacks.
5) After traversing the expression:
   - Pop all remaining operators from the stack and append them to the
6) Display the postfix expression
7) Stop

Code:

```
operators = set(['+', '-', '*', '/', '(', ')', '^'])

Priority = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}

def InfixToPostfix(expression):
    stack = []
    output = ''
    for character in expression:
        if character not in operators:
            output += character
        elif character == '(':
            stack.append('(')
        elif character == ')':
            while stack and stack[-1] != '(':
                output += stack.pop()
            stack.pop()
        else:
            while stack and stack[-1] != '(' and Priority[character] <=
                                                Priority[stack[-1]]:
                output += stack.pop()
            stack.append(character)
    while stack:
        output += stack.pop()
    return output
expression = input('Enter infix expression')
print('infix notation:', expression)
print('Postfix notation:', InfixToPostfix(expression))
```

OUTPUT:

Enter infix expression infix notation: (A + B) * (C - A) / B ^ C
Postfix Notation: AB + CA - * B C ^ /

CONCLUSION:

- The experiment successfully demonstrates
- Conversion of infix expression to Postfix using a stack.
- The use of precedence and associativity rules in conversion
- Efficient handling of parenthesis and operators.

EXPERIMENT NO: 06

TITLE: write a program to Reverse a linked list in Groups of Given size.

AIM: To implement a program in Python that:

- Reverses a singly LL in groups of a specified size

SOFTWARE REQUIRED: Python IDE

THEORY:

A singly linked list (SLL) is a linear data structure consisting of nodes, where each node contains:

- Data → Holds the value of the element
- Next Pointer → Points to the next node.

Reversing in Groups:

1) Group Size:
   - The list is divided into subgroups of the specified size.
2) Reversal Process:
   - For each group:
     - Reverse the links betw the nodes
     - Move to the next group and repeat the process.
3) Time complexity:
   - Best case: O(n) → Traverse the entire list once.
   - worst case: O(n) → Same as the best case, as all nodes are traversed.

Example:

Given the linked list

1→2→3→4→5→6→7→8→9

- Group size = 3

Output ::

3 → 2 → 1 → 6 → 5 → 4 → 9 → 8 → 7

ALGORITHM

1) Start
2) Input the group size (k).
3) Create a Singly linked list with n elements.
4) Initialize pointers :
   a. current → Points to the head of the list
   b. prev → Initially set to None
   c. next → To temporarily store the next node reference.
5) Traverse the list :
   a. For each group of size k:
      i) Reverse the links of the nodes in the current group.
      ii) Adjust the next pointer to connect to the next group.
6) Repeat the process until the entire list is reversed in groups.
7) Display the reversed list.
8) Stop